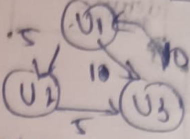
(14) SPLITWISE SIMPLIFY DEBT ALORITHM.

→ Main aim is to reduce the total number of transaction.

From	to	amount
U1	U2	5
U2	U3	5
U3	U1	10
U1	U3	1

→ when we get expense, we update the balance sheet for all users, owe & get back

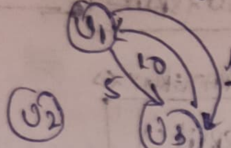
→ Total tranx = 4



→ can we able to reduce the total tranx?

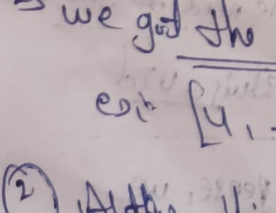
① If we see $U_1 \xrightarrow{5} U_2$ & $U_2 \xrightarrow{5} U_3$. 2 transaction.

② But U2 is acting as middleware, if we remove him, then ultimately we can ~~see~~ give $U_1 \xrightarrow{5} U_3$. 2 → 1 txn.

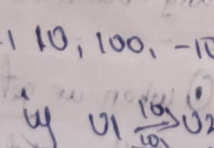


Now, $U_1 \xrightarrow{5} U_3$, $U_1 \xrightarrow{1} U_3$ & $U_3 \xrightarrow{10} U_1$. At end U1 = +4 & U3 = -4.

③ If we dont make any transaction, we may get all users owe & get back easily.



[Incoming - outgoing] U2 = 0, we can exclude them



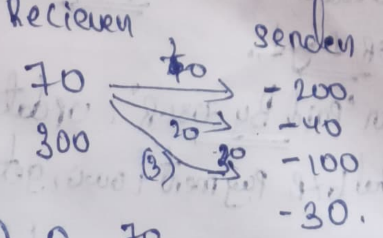
list = [4, -4].

$U_3 \xrightarrow{4} U_1$ = 1 transaction.

→ we got the list with all money excluding 0.

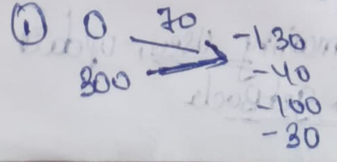
ex: [4, -4, 10, 100, -10].

② After this, if $U_1 \xrightarrow{10} U_2$, $\xrightarrow{10} U_3$, $\xrightarrow{10} U_4$ U1 want to give money to some other users, then there will be more transaction.

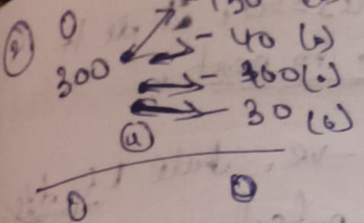


→ If sender wants to send 200 to multiple user, dont sent multiple times.

→ send to any 1 user with full money

①


→ we took 1 transaction



with total 5 txn we achieved all

Total sender = reciever > 0 valid.

---

① Convert 2D → 1D array by incoming - outgoing.

② exclude 0 from it.

[70, -200, 300, -70, -100, -30].

③ Try to give 70 → -ve int, to make transaction.
Give full amount.

④ Try all possible ways & get Min txn

 0 1 2 3
[70, -200, 70, -30]

0 → 1 0-2 0-3

[0, -130, 70, -30] [70, -200, 0, -30] [70, -200, 70, +40]

→ If we give +70 → -70, with 1 txn we made 2 of them as 0.

→ our aim is to make whole array as 0, with Min txn.

How to check valid / not. EXPONCE

① If we sum the List, we need to get sum = 0, valid
sum != 0, not valid.

→ Create Map <User, Amount> (int, int) and make from = -ve & to = +ve.
→ Create array & add all values.

→ simple take & not take approach

```
txn [][]; Map<int,int>;
for(item : txn):
 from = t[0]. to = t[1] am = t[2];
 Map.put (from, getOrDefault (from, 0) - amount);
 Map.put (to, ________ (to, 0) + amt);
}
```

```
List<Int>.
for(value){
 list.add(val);
}
```

```
dfs(arr, CI){
 if( CI == len) ret 0
 if(arr[CI]==0){
   return dfs(arr, CI+1)
 }
 MinTxn = max
 currAmt = arr[CI];
 for(ind = CI+1 -> len-1){
   nextAmt = arr[ind];
   if(currAmt x nextAmt <0){
     txn = 1+dfs(arr, ind);
     arr[ind] = currAmt + nextAmt;
     arr[ind] = currAmt;
     minTxn = min(minTxn, txn);
     if(~~currft~~ currAmt + nxtAmt ==0){
       break;
 }  }
 return MinTxn
}
```

① when a[i]==0, it is it
go to next.

② If +ve -ve, take 1+dfs,
set, notset

③ If +4 = -4 ==0. dont go
further, break it.

alternate

```
while(CI < len && a[CI]==0){
  CI++
}
if(CI == len) return 0;
```

TC = O(N!) + O(2N)

SC = O(2N).